



In []:

Prediction of Eco-Friendly Logistics Using Random Forest Classification

Problem Statement:

With growing concerns about environmental sustainability, it is important to identify logistics operations that are eco-friendly. This project aims to predict whether a logistics route is eco-friendly using a Random Forest classification model by analyzing factors such as origin and destination, vehicle type, route type, distance, package weight, traffic conditions, and carbon emissions. The model helps classify logistics operations based on their environmental impact and supports sustainable decision-making.

In []:

Steps to be followed

1. Importing Required Libraries
2. Load Dataset
3. Data Preprocessing
4. Checking Null values
5. Data Wrangling (Filling Missing values)
6. Data Visualization
7. Feature Selection (x,y)
8. Pipeline Method
9. Train-test Split
10. Random Forest Model Training and Prediction
11. Model Evaluation (Accuracy Score, Classification Report)
12. GridSearchCV
13. ROC-AUC Score and ROC-AUC Curve
14. Cross Validation
15. Threshold tuning
16. Improving Model Performance with Advanced Algorithms

Importing Libraries

```
In [2]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder,
from sklearn.metrics import classification_report, confusion_matrix, accuracy_s
import warnings
warnings.filterwarnings('ignore')
```

```
In [ ]:
```

Loading the Dataset

```
In [3]: df=pd.read_csv(r"C:\Users\HP\Downloads\carbon_emissions.csv")
df
```

Out[3]:

	Origin_Facility	Destination_City	Vehicle_Type	Route_Type	Distance_KM
0	Jakarta Fulfillment Center	Waynehaven	Electric Van (EV)	Mixed Route	80.4
1	Jakarta Fulfillment Center	East Patriciaside	NaN	Mixed Route	135.3
2	Semarang Depot	Pottershire	NaN	Urban Last Mile	9.0
3	Semarang Depot	North Christina	Drone Delivery	Urban Last Mile	11.8
4	Jakarta Fulfillment Center	Lisamouth	NaN	Inter-City	156.5
...	...	...	...	...	...
11995	Makassar Port	Perezfurt	Drone Delivery	Urban Last Mile	9.1
11996	Makassar Port	Sanchezmouth	Electric Van (EV)	NaN	67.3
11997	Medan Distribution	Lisashire	NaN	Urban Last Mile	8.5
11998	Bandung Hub	South Lori	Diesel Van (Euro 6)	Mixed Route	96.9
11999	Bandung Hub	Kristenberg	NaN	NaN	74.7

12000 rows × 9 columns

In []:

Data Preprocessing

In [4]:

```
df.head()
```

Out[4]:

	Origin_Facility	Destination_City	Vehicle_Type	Route_Type	Distance_KM	Pac
0	Jakarta Fulfillment Center	Waynehaven	Electric Van (EV)	Mixed Route	80.4	
1	Jakarta Fulfillment Center	East Patriciaside	NaN	Mixed Route	135.3	
2	Semarang Depot	Pottershire	NaN	Urban Last Mile	9.0	
3	Semarang Depot	North Christina	Drone Delivery	Urban Last Mile	11.8	
4	Jakarta Fulfillment Center	Lisamouth	NaN	Inter-City	156.5	

In [5]: `df.tail()`

Out[5]:

	Origin_Facility	Destination_City	Vehicle_Type	Route_Type	Distance_KM	
11995	Makassar Port	Perezfurt	Drone Delivery	Urban Last Mile	9.1	
11996	Makassar Port	Sanchezmouth	Electric Van (EV)	NaN	67.3	
11997	Medan Distribution	Lisashire	NaN	Urban Last Mile	8.5	
11998	Bandung Hub	South Lori	Diesel Van (Euro 6)	Mixed Route	96.9	
11999	Bandung Hub	Kristenberg	NaN	NaN	74.7	

In [6]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12000 entries, 0 to 11999
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Origin_Facility                       12000 non-null  object
1   Destination_City                      12000 non-null  object
2   Vehicle_Type                          8689 non-null   object
3   Route_Type                            10224 non-null  object
4   Distance_KM                           12000 non-null  float64
5   Package_Weight_KG                    10924 non-null  float64
6   Traffic_Conditions                    7507 non-null   object
7   Carbon_Emission_kgCO2e               12000 non-null  float64
8   Is_Eco_Friendly                      12000 non-null  int64
dtypes: float64(3), int64(1), object(5)
memory usage: 843.9+ KB
```

```
In [7]: df.describe()
```

```
Out[7]:
```

	Distance_KM	Package_Weight_KG	Carbon_Emission_kgCO2e	Is_Eco_Frien
count	12000.000000	10924.000000	12000.000000	12000.000
mean	102.851408	401.507232	71.994260	0.427
std	163.538763	1073.095927	180.524361	0.494
min	1.000000	0.500000	0.000000	0.000
25%	11.600000	8.800000	0.093000	0.000
50%	37.500000	17.400000	4.150000	0.000
75%	114.600000	26.100000	24.091250	1.000
max	799.900000	4985.500000	1280.173000	1.000

```
In [ ]:
```

Checking Null Values

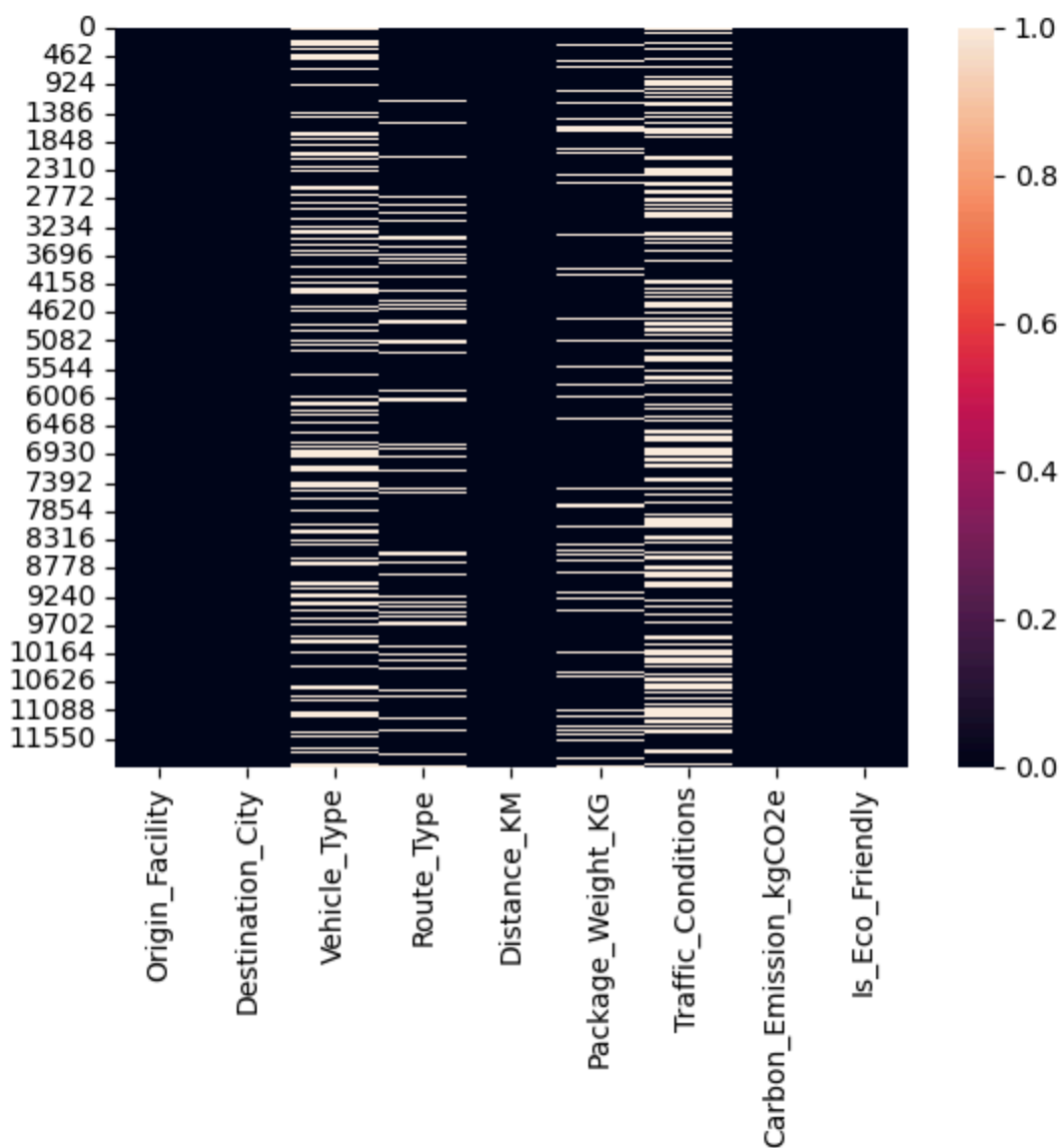
Missing values are identified to ensure data quality before training the model.

```
In [8]: df.isnull().sum()
```

```
Out[8]: Origin_Facility          0
Destination_City                0
Vehicle_Type                    3311
Route_Type                      1776
Distance_KM                     0
Package_Weight_KG               1076
Traffic_Conditions              4493
Carbon_Emission_kgCO2e          0
Is_Eco_Friendly                 0
dtype: int64
```

```
In [9]: sns.heatmap(df.isnull())
```

```
Out[9]: <Axes: >
```



```
In [10]: df.isna().mean()*100
```

```
Out[10]: Origin_Facility      0.000000
Destination_City      0.000000
Vehicle_Type      27.591667
Route_Type      14.800000
Distance_KM      0.000000
Package_Weight_KG      8.966667
Traffic_Conditions      37.441667
Carbon_Emission_kgCO2e      0.000000
Is_Eco_Friendly      0.000000
dtype: float64
```

```
In [11]: df.dtypes
```

```
Out[11]: Origin_Facility      object
         Destination_City    object
         Vehicle_Type        object
         Route_Type          object
         Distance_KM         float64
         Package_Weight_KG   float64
         Traffic_Conditions   object
         Carbon_Emission_kgCO2e float64
         Is_Eco_Friendly      int64
         dtype: object
```

```
In [12]: df.columns
```

```
Out[12]: Index(['Origin_Facility', 'Destination_City', 'Vehicle_Type', 'Route_Type',
               'Distance_KM', 'Package_Weight_KG', 'Traffic_Conditions',
               'Carbon_Emission_kgCO2e', 'Is_Eco_Friendly'],
              dtype='object')
```

Data Wrangling (Filling Null/Missing Values)

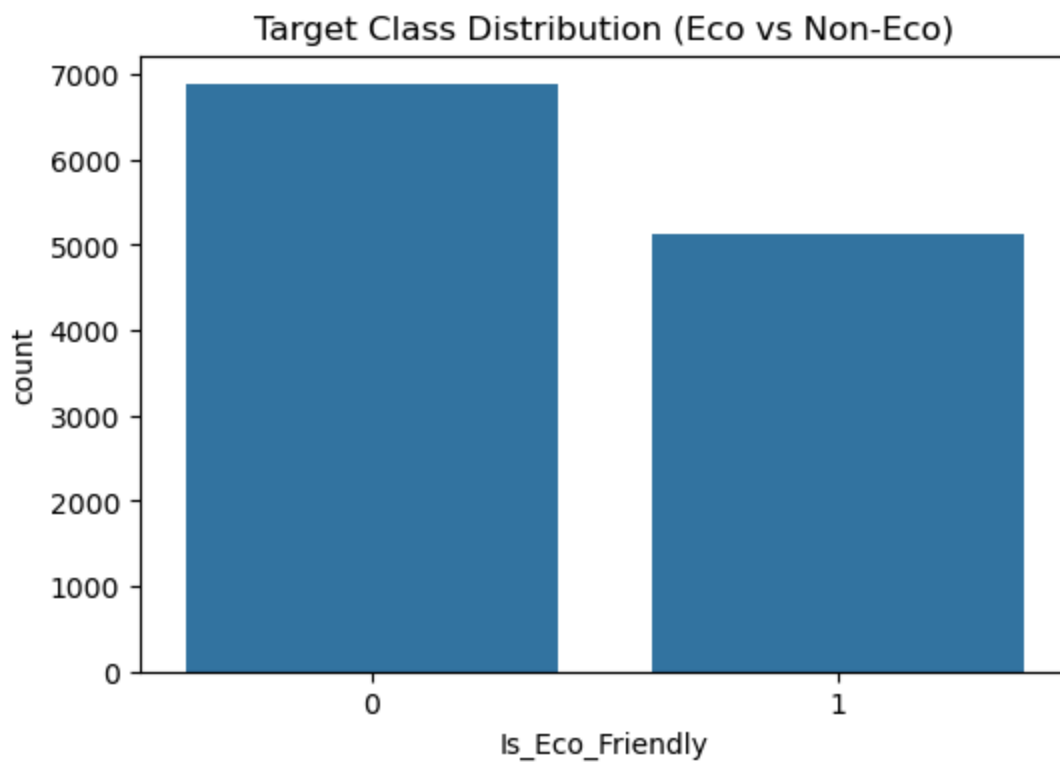
Some numerical and categorical features contain missing values which will be handled using median and mode imputation respectively.

```
In [13]: num_cols = ['Package_Weight_KG']
         for col in num_cols:
             df[col].fillna(df[col].median(), inplace=True)
```

```
In [14]: cat_cols = ['Vehicle_Type', 'Route_Type', 'Traffic_Conditions']
         for col in cat_cols:
             df[col].fillna(df[col].mode()[0], inplace=True)
```

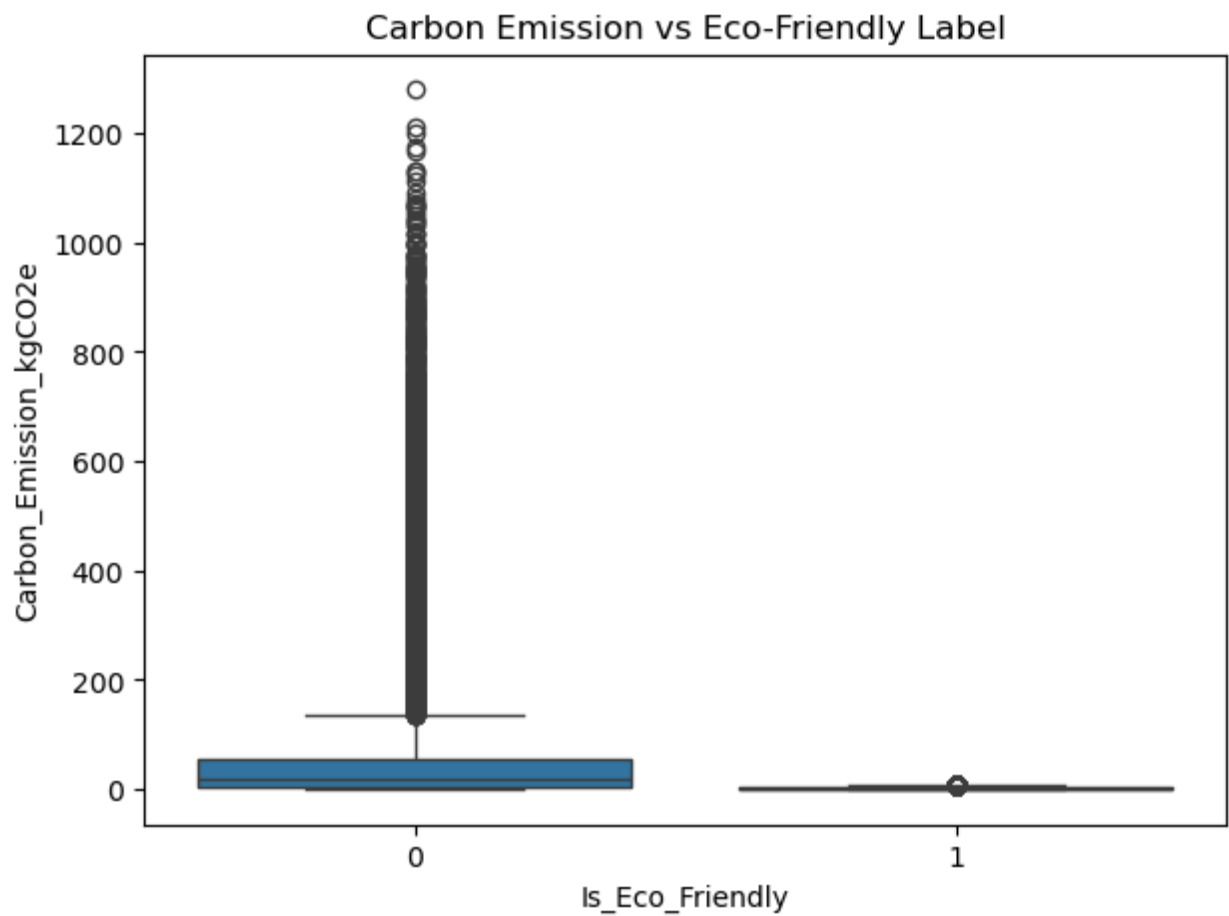
Data Visualization

```
In [15]: plt.figure(figsize=(6,4))
         sns.countplot(x='Is_Eco_Friendly', data=df)
         plt.title("Target Class Distribution (Eco vs Non-Eco)")
         plt.show()
```



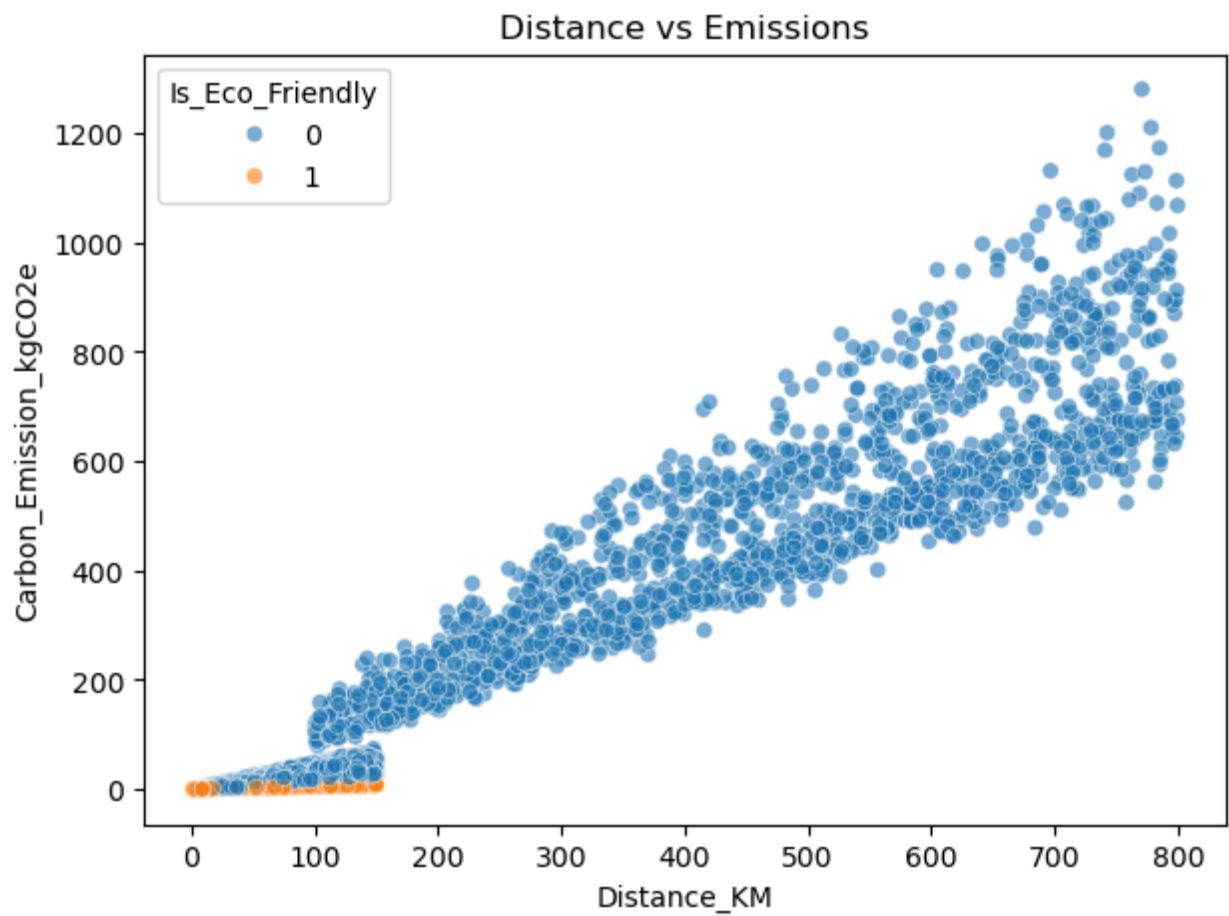
Eco-friendly deliveries are slightly fewer than non-eco deliveries, indicating a moderately balanced dataset.

```
In [16]: plt.figure(figsize=(7,5))  
sns.boxplot(x='Is_Eco_Friendly', y='Carbon_Emission_kgCO2e', data=df)  
plt.title("Carbon Emission vs Eco-Friendly Label")  
plt.show()
```

Eco-friendly deliveries have significantly lower carbon emissions compared to non-eco deliveries.

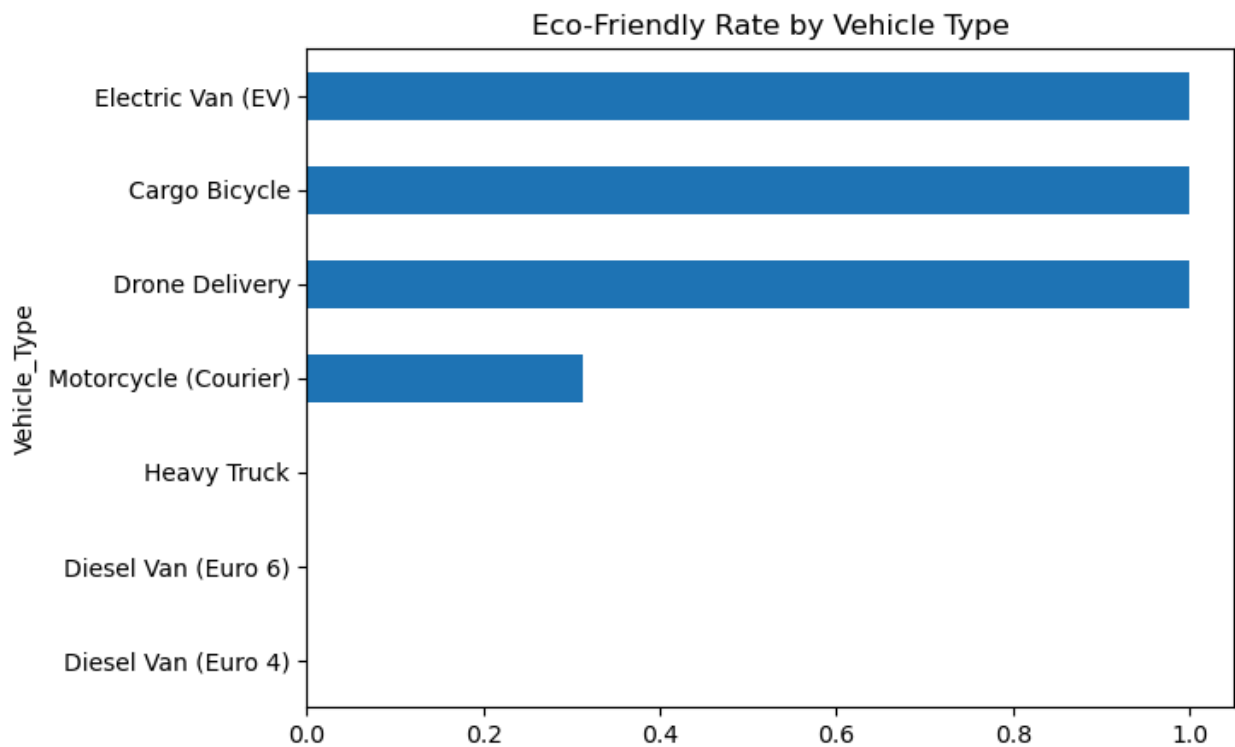
```
In [17]: plt.figure(figsize=(7,5))
sns.scatterplot(
    x='Distance_KM',
    y='Carbon_Emission_kgCO2e',
    hue='Is_Eco_Friendly',
    data=df,
    alpha=0.6
)
plt.title("Distance vs Emissions")
plt.show()
```



Carbon emissions increase as delivery distance increases, showing a strong positive relationship.

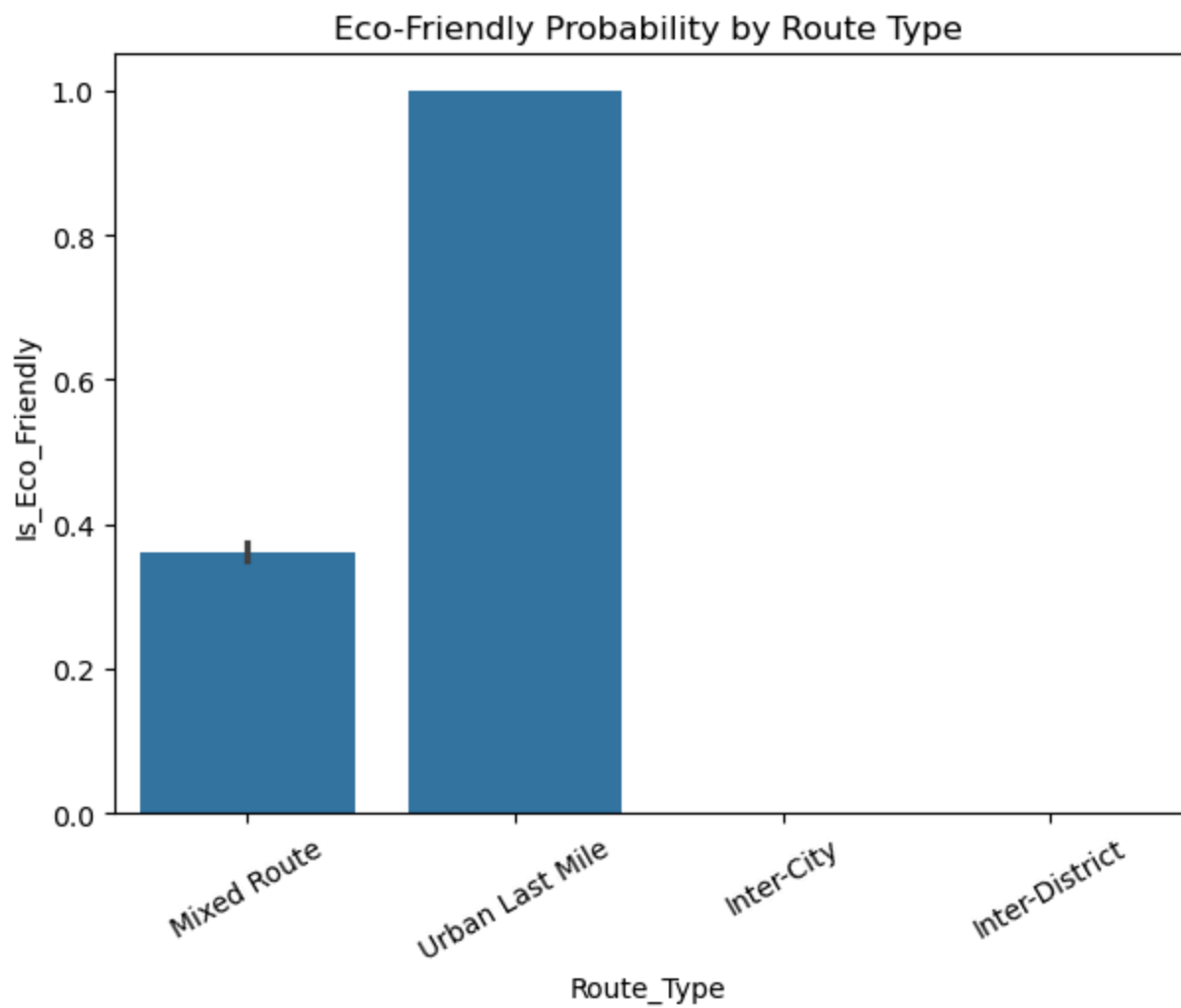
```
In [18]: eco_rate = df.groupby('Vehicle_Type')['Is_Eco_Friendly'].mean().sort_values()

eco_rate.plot(kind='barh', figsize=(7,5))
plt.title("Eco-Friendly Rate by Vehicle Type")
plt.show()
```



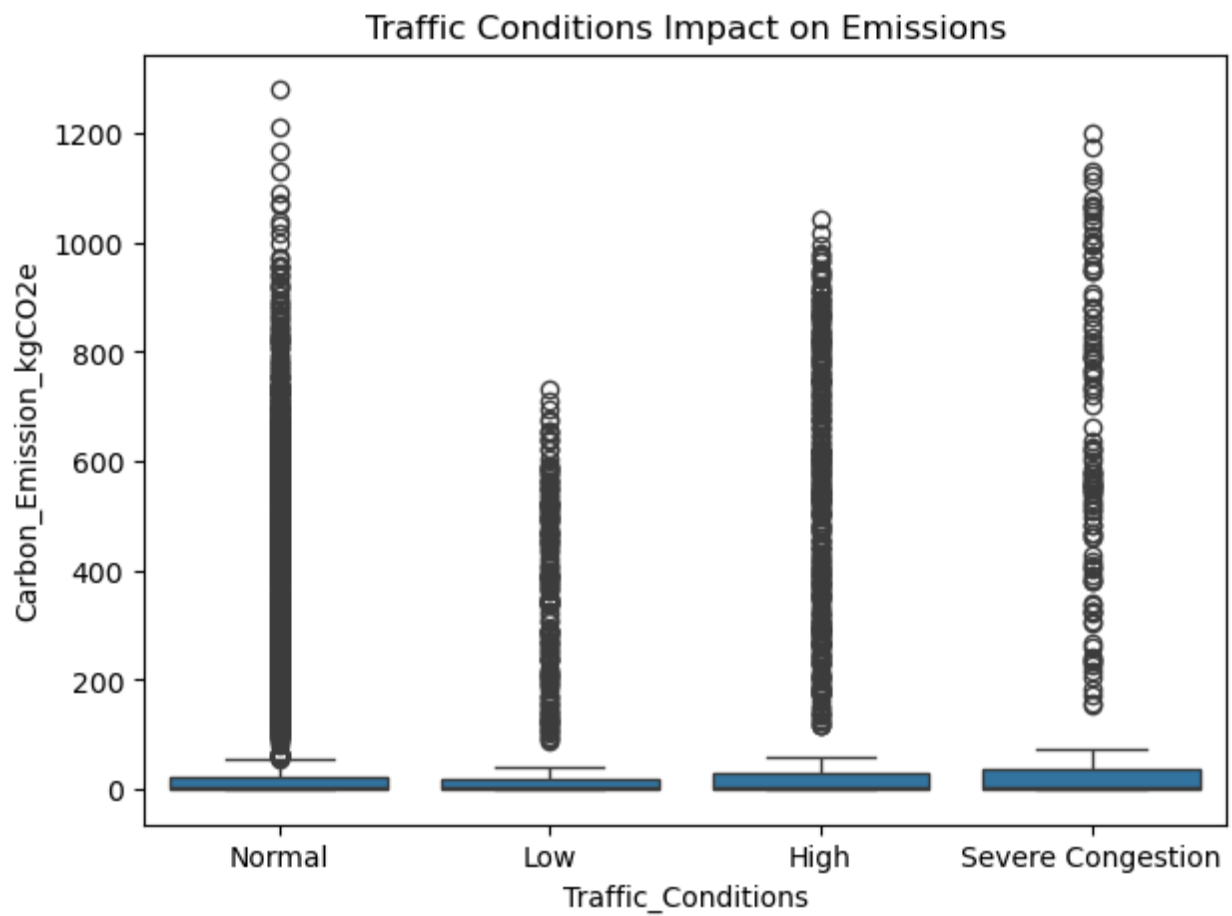
Electric vans, cargo bicycles, and drone deliveries show the highest eco-friendly delivery rates.

```
In [19]: plt.figure(figsize=(7,5))
sns.barplot(
    x='Route_Type',
    y='Is_Eco_Friendly',
    data=df,
    estimator='mean')
plt.title("Eco-Friendly Probability by Route Type")
plt.xticks(rotation=30)
plt.show()
```



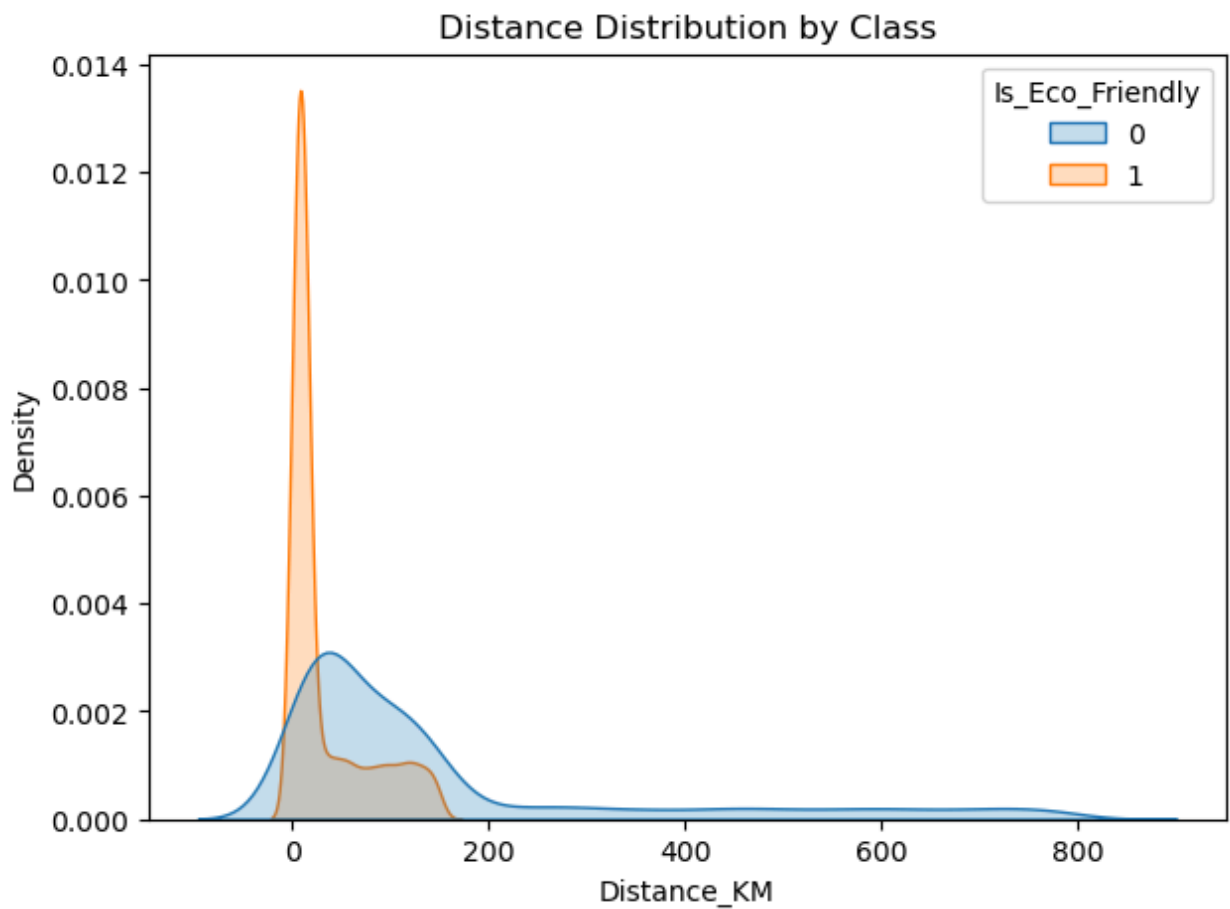
Urban last-mile routes have the highest probability of eco-friendly deliveries.

```
In [20]: plt.figure(figsize=(7,5))
sns.boxplot(
    x='Traffic_Conditions',
    y='Carbon_Emission_kgC02e',
    data=df
)
plt.title("Traffic Conditions Impact on Emissions")
plt.show()
```



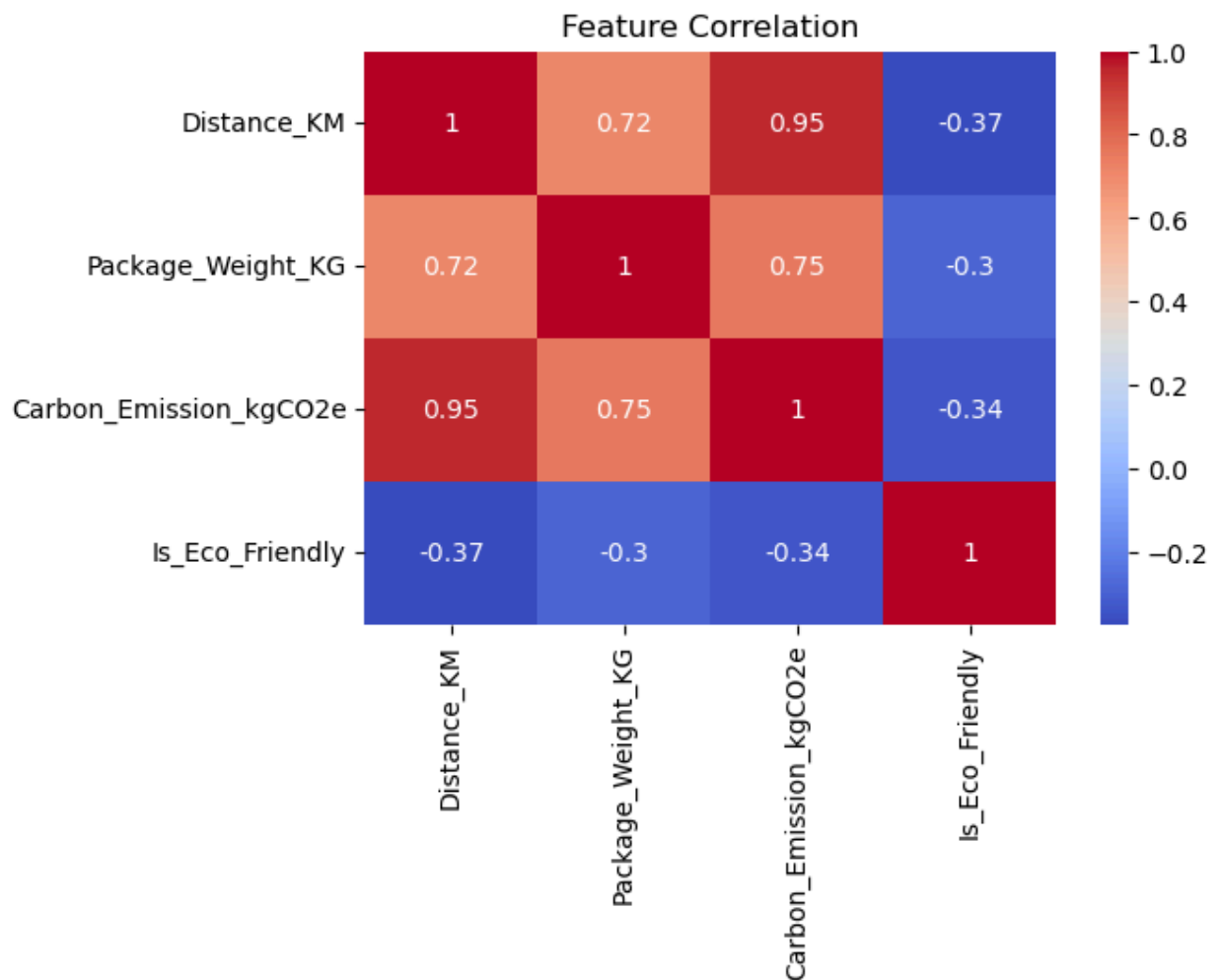
Carbon emissions tend to increase under severe traffic congestion conditions.

```
In [21]: plt.figure(figsize=(7,5))
sns.kdeplot(
    data=df,
    x='Distance_KM',
    hue='Is_Eco_Friendly',
    fill=True
)
plt.title("Distance Distribution by Class")
plt.show()
```



Eco-friendly deliveries mostly occur over shorter distances compared to non-eco deliveries.

```
In [22]: plt.figure(figsize=(6,4))
sns.heatmap(
    df[['Distance_KM', 'Package_Weight_KG', 'Carbon_Emission_kgCO2e', 'Is_Eco_Fri
    annot=True,
    cmap='coolwarm'
])
plt.title("Feature Correlation")
plt.show()
```



Distance and carbon emissions show a very strong positive correlation.

```
In [23]: import plotly.express as px

fig = px.scatter(
    df,
    x='Distance_KM',
    y='Carbon_Emission_kgCO2e',
    color='Is_Eco_Friendly',
    title="Distance vs Emission"
)
fig.show()
```

Carbon emissions increase as delivery distance increases, showing a strong positive relationship.

In []:

```
In [24]: fig = px.histogram(  
    df,  
    x='Carbon_Emission_kgCO2e',  
    color='Is_Eco_Friendly',  
    nbins=40,  
    title="Emission Distribution by Class"  
)  
fig.show()
```


Non-eco deliveries dominate the higher emission ranges.

```
In [25]: fig = px.scatter(  
    df,  
    x='Distance_KM',  
    y='Package_Weight_KG',  
    size='Carbon_Emission_kgCO2e',  
    color='Is_Eco_Friendly',  
    title="Distance vs Weight vs Emission"  
)  
fig.show()
```

Higher package weight and longer distances generally lead to higher carbon emissions.

```
In [26]: df.columns
```

```
Out[26]: Index(['Origin_Facility', 'Destination_City', 'Vehicle_Type', 'Route_Type',  
              'Distance_KM', 'Package_Weight_KG', 'Traffic_Conditions',  
              'Carbon_Emission_kgCO2e', 'Is_Eco_Friendly'],  
              dtype='object')
```

```
In [ ]:
```

Feature Selection (x, y)

```
In [27]: x = df.drop(['Is_Eco_Friendly', 'Carbon_Emission_kgCO2e'], axis=1)  
y = df['Is_Eco_Friendly']  
cat_cols = x.select_dtypes(include='object').columns  
num_cols = x.select_dtypes(exclude='object').columns
```

Pipeline Method

The pipeline integrates preprocessing (One-Hot Encoding for categorical variables) and model training into a single streamlined workflow for efficient and consistent model building.

```
In [28]: from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```
In [29]: preprocessor = ColumnTransformer(
    transformers=[('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols),
                  ('num', 'passthrough', num_cols)])
```

```
In [30]: rf_pipeline = Pipeline(steps=[('preprocessing', preprocessor), ('model', RandomForestClassifier(
    n_estimators=200, random_state=42, n_jobs=-1))])
```

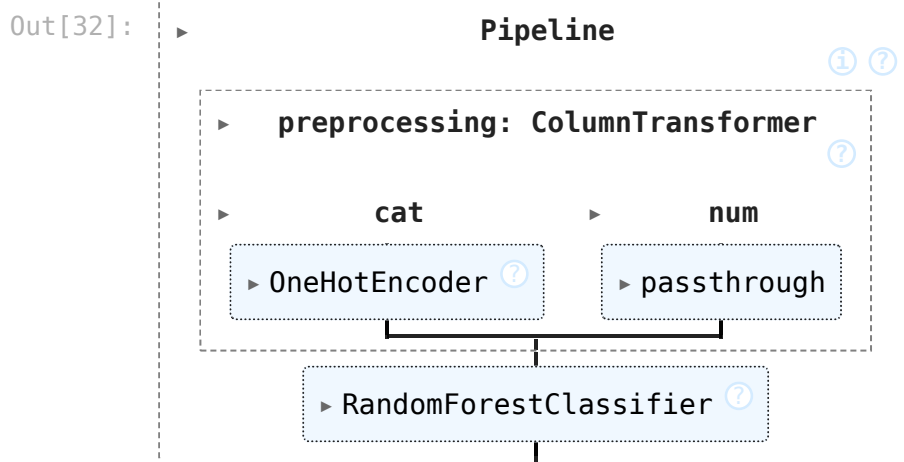
Train-Test Split

The dataset is divided into 70% training data and 30% testing data.

```
In [31]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
```

```
In [43]: x_train = pd.get_dummies(x_train)
x_test = pd.get_dummies(x_test)
x_train, x_test = x_train.align(x_test, join='left', axis=1, fill_value=0)
```

```
In [32]: rf_pipeline.fit(x_train, y_train)
```



Model Prediction

```
In [33]: y_pred = rf_pipeline.predict(x_test)
y_prob = rf_pipeline.predict_proba(x_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.9491666666666667

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.99	0.96	2093
1	0.99	0.89	0.94	1507
accuracy			0.95	3600
macro avg	0.96	0.94	0.95	3600
weighted avg	0.95	0.95	0.95	3600

In []:

GridSearchCV

GridSearchCV tests multiple combinations of hyperparameters to find the best K value and distance metric.

```
In [34]: param_grid = {
    'model__n_estimators': [100, 200, 300],
    'model__max_depth': [None, 10, 20],
    'model__min_samples_split': [2, 5]
}

grid = GridSearchCV(rf_pipeline, param_grid, cv=5, scoring='f1', n_jobs=-1)
grid.fit(x_train, y_train)
print("Best Params:", grid.best_params_)
```

Best Params: {'model__max_depth': None, 'model__min_samples_split': 5, 'model__n_estimators': 200}

ROC-AUC

The Receiver Operating Characteristic (ROC) curve is used to evaluate the performance of a classification model at different classification thresholds. The Area Under the Curve (AUC) provides a single metric that summarizes the model's

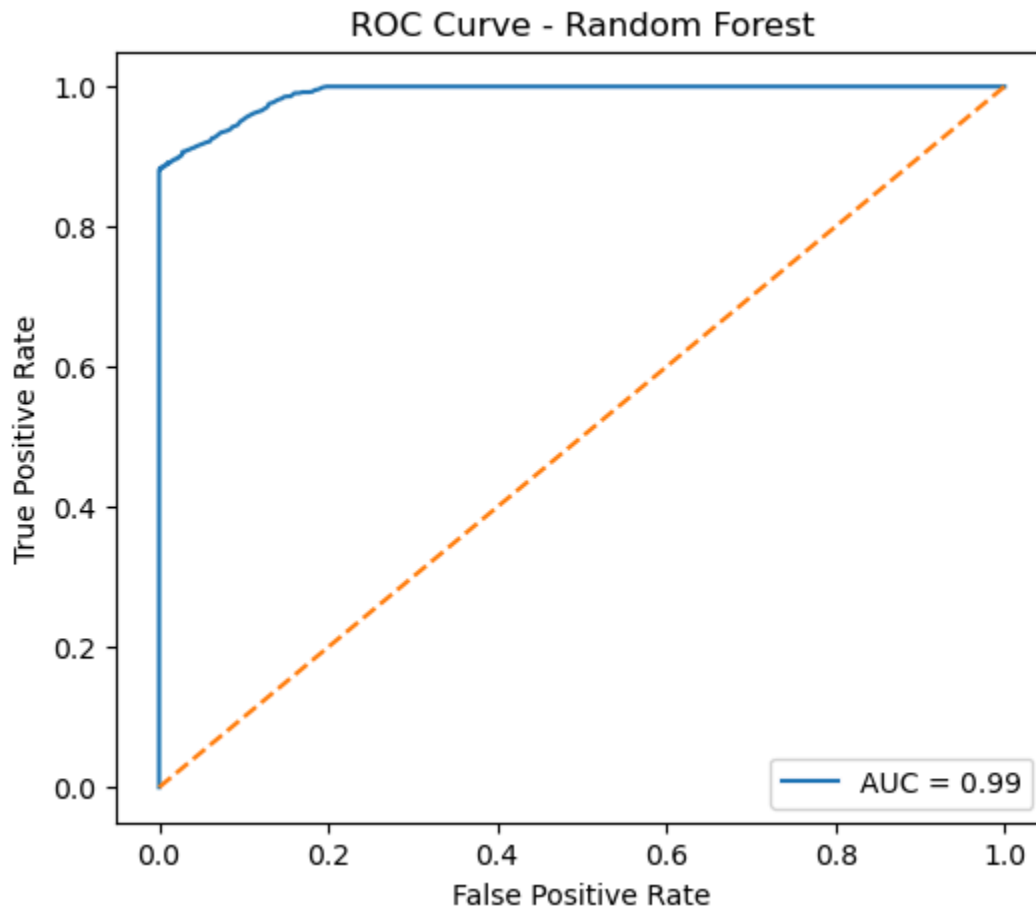
ability to distinguish between the classes. A higher AUC score indicates better classification performance.

```
In [35]: fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc_score = roc_auc_score(y_test, y_prob)

print("ROC AUC Score:", auc_score)
```

ROC AUC Score: 0.9898964887857303

```
In [36]: plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"AUC = {auc_score:.2f}")
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Random Forest")
plt.legend()
plt.show()
```



```
In [ ]:
```

```
In [37]: print("Train AUC:", roc_auc_score(y_train, rf_pipeline.predict_proba(x_train)[
print("Test AUC:", roc_auc_score(y_test, y_prob))
```

Train AUC: 1.0
Test AUC: 0.9898964887857303

In []:

Cross Validation

Cross-validation evaluates model stability by training and testing the model on multiple data splits.

```
In [38]: cv_auc = cross_val_score(rf_pipeline, x, y, cv=5, scoring='roc_auc', n_jobs=-1)
print("Mean CV AUC:", cv_auc.mean())
```

Mean CV AUC: 0.9900578306982645

```
In [39]: thresholds = np.arange(0.1, 0.9, 0.05)
for t in thresholds:
    y_custom = (y_prob >= t).astype(int)
    print(t, accuracy_score(y_test, y_custom))
```

```
0.1 0.8980555555555556
0.15000000000000002 0.9133333333333333
0.20000000000000004 0.9255555555555556
0.25000000000000006 0.9341666666666667
0.30000000000000004 0.9427777777777778
0.35000000000000001 0.9444444444444444
0.400000000000000013 0.9466666666666667
0.45000000000000007 0.9486111111111111
0.50000000000000001 0.9491666666666667
0.55000000000000002 0.95
0.60000000000000002 0.9505555555555556
0.65000000000000001 0.95
0.70000000000000002 0.9494444444444444
0.75000000000000002 0.9494444444444444
0.80000000000000002 0.9483333333333334
0.85000000000000002 0.9466666666666667
```

In []:

Threshold tuning

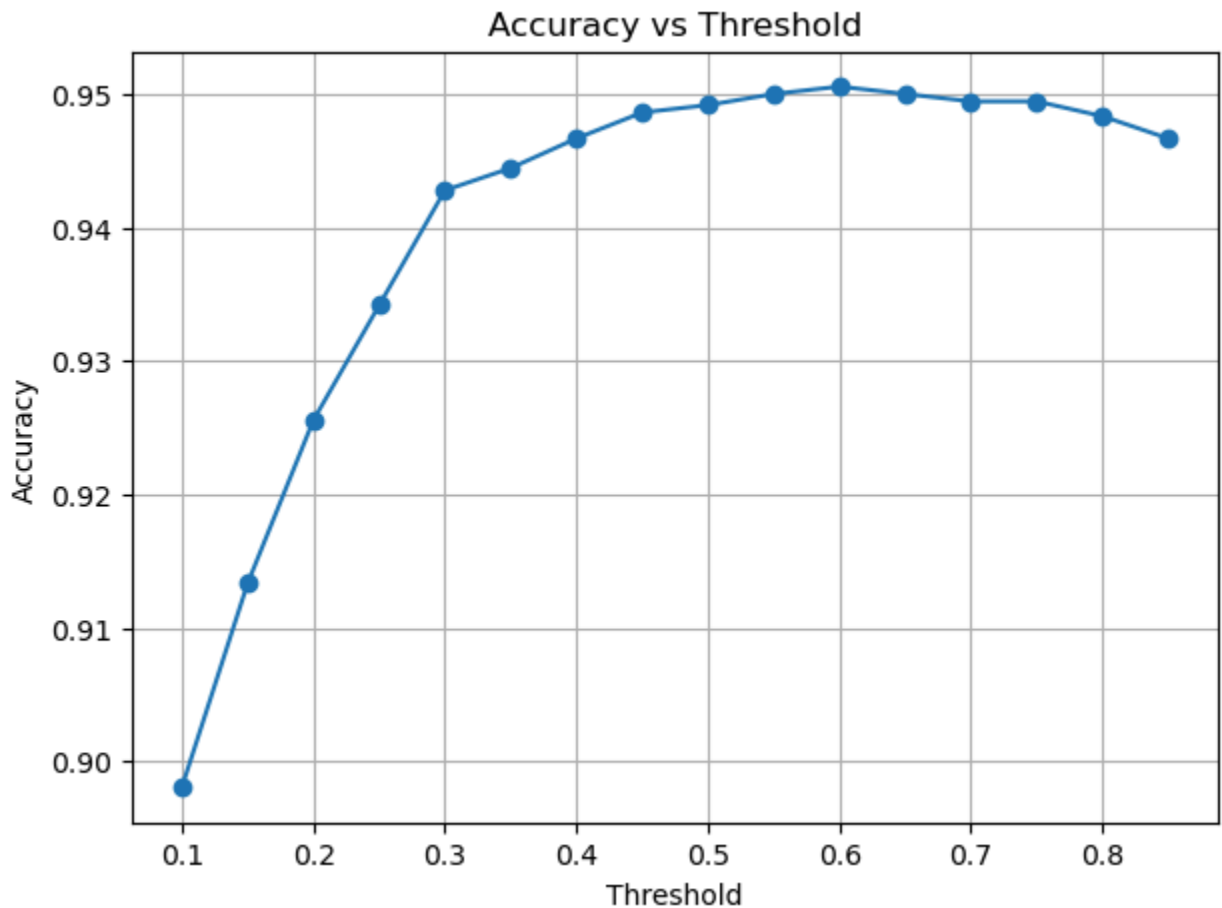
Threshold tuning evaluates different probability cutoffs to find the value that gives the best model accuracy.

```
In [40]: thresholds = np.arange(0.1, 0.9, 0.05)
accuracies = []

for t in thresholds:
    y_custom = (y_prob >= t).astype(int)
```

```
acc = accuracy_score(y_test, y_custom)
accuracies.append(acc)
```

```
In [41]: plt.figure(figsize=(7,5))
plt.plot(thresholds, accuracies, marker='o')
plt.xlabel("Threshold")
plt.ylabel("Accuracy")
plt.title("Accuracy vs Threshold")
plt.grid(True)
plt.show()
```



Model accuracy improves as the threshold increases and reaches its peak around 0.6, after which the accuracy slightly decreases.

```
In [ ]:
```

Improving Model Performance with Advanced Algorithms

```
In [ ]:
```

```
In [49]: import warnings
warnings.filterwarnings("ignore")

from sklearn.preprocessing import LabelEncoder

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier, AdaBoostClassifier

from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# ENCODE TARGET
le = LabelEncoder()
y_train = le.fit_transform(y_train)
y_test = le.transform(y_test)

# EVALUATION FUNCTION
def evaluate(y_true, y_pred):
    return (
        accuracy_score(y_true, y_pred),
        precision_score(y_true, y_pred, average='weighted'),
        recall_score(y_true, y_pred, average='weighted'),
        f1_score(y_true, y_pred, average='weighted')
    )

results = []

# LOGISTIC REGRESSION
lr = LogisticRegression(C=1, max_iter=1000)
lr.fit(x_train, y_train)
y_pred = lr.predict(x_test)
results.append(["Logistic Regression", *evaluate(y_test, y_pred)])

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(x_train, y_train)
y_pred = knn.predict(x_test)
results.append(["KNN", *evaluate(y_test, y_pred)])

# SVM
svm = SVC(C=1, kernel='rbf')
svm.fit(x_train, y_train)
y_pred = svm.predict(x_test)
results.append(["SVM", *evaluate(y_test, y_pred)])

# DECISION TREE
dt = DecisionTreeClassifier(max_depth=10)
dt.fit(x_train, y_train)
```



```

y_pred = dt.predict(x_test)
results.append(["Decision Tree", *evaluate(y_test, y_pred)])

# GRADIENT BOOSTING
gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1)
gb.fit(x_train, y_train)
y_pred = gb.predict(x_test)
results.append(["Gradient Boosting", *evaluate(y_test, y_pred)])

# ADABOOST
ab = AdaBoostClassifier(n_estimators=50)
ab.fit(x_train, y_train)
y_pred = ab.predict(x_test)
results.append(["AdaBoost", *evaluate(y_test, y_pred)])

# XGBOOST
xgb = XGBClassifier(n_estimators=100, learning_rate=0.1, use_label_encoder=False)
xgb.fit(x_train, y_train)
y_pred = xgb.predict(x_test)
results.append(["XGBoost", *evaluate(y_test, y_pred)])

# LIGHTGBM
lgb = LGBMClassifier(n_estimators=50, learning_rate=0.1, verbose=-1)
lgb.fit(x_train, y_train)
y_pred = lgb.predict(x_test)
results.append(["LightGBM", *evaluate(y_test, y_pred)])

# CATBOOST
cb = CatBoostClassifier(iterations=50, learning_rate=0.1, verbose=0)
cb.fit(x_train, y_train)
y_pred = cb.predict(x_test)
results.append(["CatBoost", *evaluate(y_test, y_pred)])

# RESULT TABLE
columns = ["Model", "Accuracy", "Precision", "Recall", "F1 Score"]

df_cls = pd.DataFrame(results, columns=columns)
df_cls = df_cls.sort_values(by="Accuracy", ascending=False)

display(df_cls)

```

	Model	Accuracy	Precision	Recall	F1 Score
3	Decision Tree	0.950833	0.954283	0.950833	0.950327
8	CatBoost	0.950278	0.953011	0.950278	0.949825
5	AdaBoost	0.950000	0.951538	0.950000	0.949660
4	Gradient Boosting	0.950000	0.951538	0.950000	0.949660
6	XGBoost	0.949444	0.952037	0.949444	0.948995
0	Logistic Regression	0.944167	0.948623	0.944167	0.943487
7	LightGBM	0.943056	0.944282	0.943056	0.942695
1	KNN	0.862222	0.863891	0.862222	0.860622
2	SVM	0.664722	0.693571	0.664722	0.665759

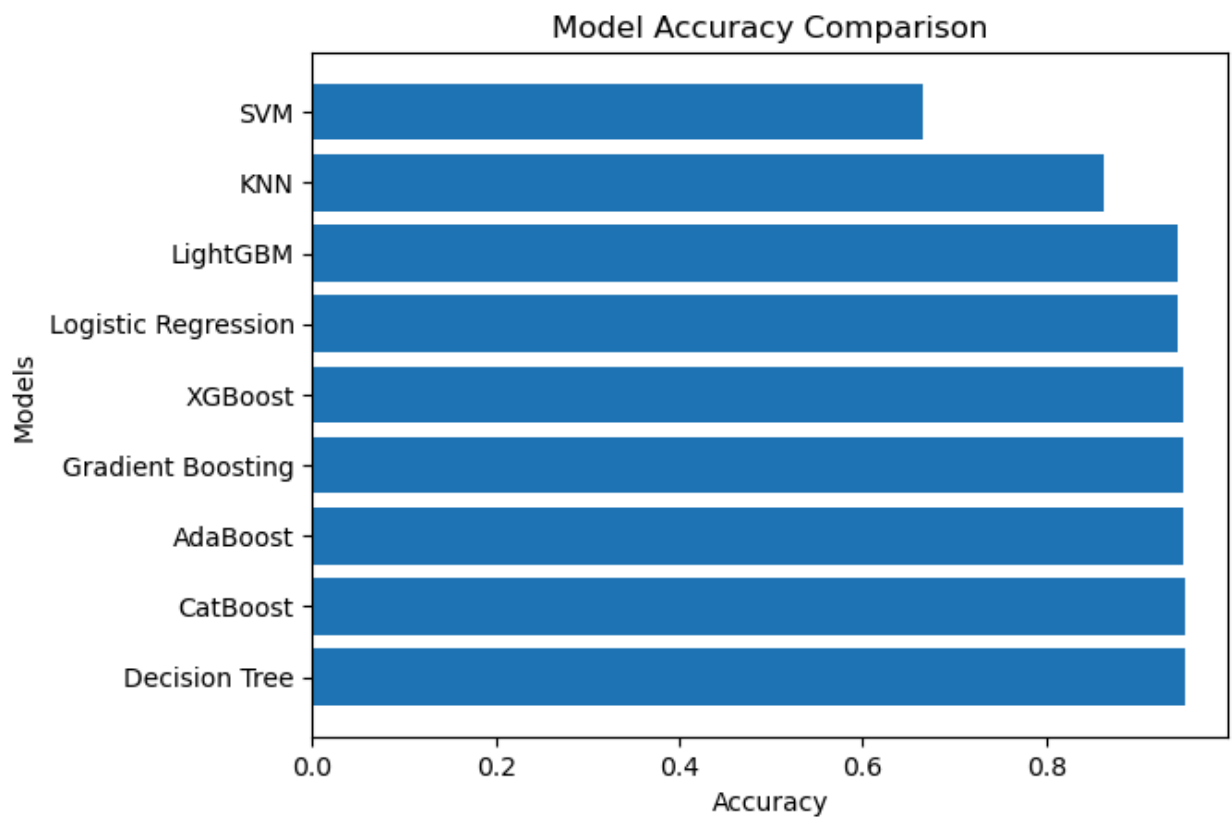
In []:

```
In [51]: plt.figure()

plt.barh(df_cls["Model"], df_cls["Accuracy"])

plt.xlabel("Accuracy")
plt.ylabel("Models")
plt.title("Model Accuracy Comparison")

plt.show()
```



Ensemble and boosting models (XGBoost, Gradient Boosting, LightGBM, CatBoost) perform the best, while SVM shows the lowest accuracy among all models.

In []:

In []:

In []:

In []:

In []:

In []:

In []: